

REPUBLIC OF TÜRKİYE

**YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMISTRY AND METALLURGY
DEPARTMENT OF MATHEMATICAL ENGINEERING**



Multidisciplinary Design Project

Design and Implementation of an Automated Exam Generation System

Demir KİRACIOĞLU-22058007

Advisor
Assoc. Prof. Dr. Melih ÇINAR

Istanbul, 2026

Contents

	Page
FOREWORD	iii
ABSTRACT	iv
ÖZET	v
LIST OF FIGURES	vii
1 INTRODUCTION	1
2 LITERATURE REVIEW	2
2.1 Traditional vs. Automated Assessment	2
2.2 Algorithmic Approaches in Exam Generation	2
3 METHODOLOGY AND THEORETICAL BACKGROUND	3
3.1 Difficulty Weighting Model	3
3.2 Monte Carlo Sampling Approach	3
3.3 Component Level Implementation Details	4
3.3.1 Data Persistence Layer (database.py)	4
3.3.2 Data Ingestion Module (importer.py)	4
3.3.3 Document Construction Module (exporter.py)	6
3.3.4 Object Models (models.py)	6
4 APPLICATION AND SYSTEM DESIGN	8
4.1 Database Design and Schema	8
4.2 User Interface Implementation	8
4.3 Algorithmic Implementation	10
4.4 Document Generation (Export Module)	11
5 CONCLUSION	13
5.1 Future Work	13
RESUME	15

FOREWORD

I would like to thank my advisor, Assoc. Prof. Dr. Melih CINAR, and my dear family for their patience and support.

ABSTRACT

This study presents a Python, Streamlit, and SQLite-based automation system designed to address time inefficiencies and pedagogical imbalances in exam preparation. By categorizing questions into three difficulty levels and utilizing numerical weights (1, 3, 5) with a Monte Carlo simulation-based selection method, the system ensures statistical alignment with target difficulty levels. Key features include multi-format (PDF/Word) exportation and an archiving system for historical examination sessions. The application significantly increases operational efficiency and minimizes human error by automating the examination workflow.

Keywords: Automated Exam Generation, Monte Carlo Simulation, Question Bank Management, Python, Pedagogical Measurement.

ÖZET

Bu çalışmada, manuel sınav hazırlama süreçlerindeki zaman kaybı ve pedagojik denge sorunlarını gidermek amacıyla Python, Streamlit ve SQLite tabanlı bir otomasyon sistemi geliştirilmiştir. Sistem; kolay, orta ve zor olarak sınıflandırılan soruların seçiminde, hedef zorluk seviyesine istatistiksel uyum sağlamak için sayısal ağırlıklar (1, 3, 5) ve Monte Carlo simülasyonu tabanlı bir yöntem kullanmaktadır. Oluşturulan sınavların PDF ve Word formatlarında dışa aktarılmasına ve arşivlenmesine olanak tanıyan bu uygulama ile sınav hazırlama süreçleri otomatize edilerek operasyonel verimlilik artırılmış ve insan kaynaklı hatalar minimize edilmiştir.

Anahtar Kelimeler: Otomatik Sınav Oluşturma, Monte Carlo Simülasyonu, Soru Bankası Yönetimi, Python, Pedagojik Ölçme.

ABBREVIATIONS

API: Application Programming Interface (Uygulama Programlama Arayüzü)
CSV: Comma-Separated Values (Virgülle Ayrılmış Değerler)
DB: Database (Veritabanı)
DOCX: Microsoft Word Open XML Document
JSON: JavaScript Object Notation
LLM: Large Language Model (Büyük Dil Modeli)
MC: Monte Carlo
MDP: Multidisciplinary Design Project (Çok Disiplinli Tasarım Projesi)
PDF: Portable Document Format (Taşınabilir Belge Formatı)
RAM: Random Access Memory (Rastgele Erişimli Bellek)
SQL: Structured Query Language (Yapılandırılmış Sorgu Dili)
UI: User Interface (Kullanıcı Arayüzü)

List of Figures

3.1	System Logic Flowchart: Visual representation of the question selection and export process.	7
4.1	The Manual Data Entry Form. This interface allows instructors to categorize questions by subject and difficulty dynamically.	9
4.2	The Bulk Upload Interface. Users can drag and drop Excel files to populate the database instantly.	9
4.3	The final output generated by the system, showing the question structure and format.	12

1 INTRODUCTION

In modern education systems, managing a large repository of questions and generating balanced examinations manually is a time-consuming and error-prone process. This project proposes an automated "Question Bank and Exam Generation System" designed to streamline these tasks. The primary objective is to develop a tool that allows educators to categorize questions by difficulty and subject, while utilizing a computational approach to ensure that generated exams meet specific pedagogical standards. By leveraging Python's powerful data processing libraries and the Streamlit framework, this system provides a reactive web interface for dynamic exam creation and export.

2 LITERATURE REVIEW

The transition from traditional paper-based examinations to Computer-Based Testing (CBT) systems has been a significant trend in educational technology over the last two decades. While many Learning Management Systems (LMS) such as Moodle, Canvas, and Blackboard offer exam modules, they often focus on static question delivery rather than intelligent, balanced exam generation.

2.1 Traditional vs. Automated Assessment

In traditional settings, educators manually select questions from a pool. This process is susceptible to human bias; an instructor might inadvertently select too many difficult questions or cluster questions from a single topic. Automated systems aim to eliminate this bias. However, most basic random selection algorithms (e.g., simple SQL randomization) fail to guarantee a pedagogical difficulty curve. A purely random selection might result in an exam that is statistically too easy or too hard.

2.2 Algorithmic Approaches in Exam Generation

Advanced systems employ various algorithms to solve the "Test Assembly Problem":

- **Linear Programming:** Optimization techniques that treat exam generation as a knapsack problem. While precise, they are computationally expensive for large datasets.
- **Genetic Algorithms:** Evolutionary approaches that "evolve" a question set. These are effective but complex to implement for simple desktop applications.
- **Monte Carlo Methods:** As implemented in this project, Monte Carlo simulations offer a balance between computational efficiency and statistical accuracy. By repeatedly sampling the question pool and evaluating the "fitness" (difficulty average) of each sample, the system can converge on an optimal exam structure without the overhead of heavy optimization solvers [1].

This project differentiates itself from standard LMS plugins by integrating a lightweight, Python-based Monte Carlo engine directly into a user-friendly Streamlit interface.

3 METHODOLOGY AND THEORETICAL BACKGROUND

This section details the mathematical and algorithmic foundations of the automated exam generation system. To ensure pedagogical consistency, the system utilizes a quantitative approach to difficulty assessment.

3.1 Difficulty Weighting Model

The system categorizes questions into three levels. To perform computational analysis, these labels are mapped to a numerical scale $W = \{1, 3, 5\}$ [2]. The difficulty weight w_i for any given question q_i is defined as:

$$w_i = \begin{cases} 1, & \text{if } q_i \text{ is Easy} \\ 3, & \text{if } q_i \text{ is Medium} \\ 5, & \text{if } q_i \text{ is Hard} \end{cases} \quad (3.1)$$

For a generated exam set S containing n questions, the average difficulty μ_S is calculated as [1]:

$$\mu_S = \frac{1}{n} \sum_{i=1}^n w_i \quad (3.2)$$

3.2 Monte Carlo Sampling Approach

A stochastic approach based on the Monte Carlo method is implemented to achieve statistical alignment with target difficulty ranges. The algorithm iteratively samples questions until the average difficulty μ_S falls within the predefined boundaries:

- **Easy Exam Target:** $1.0 \leq \mu_S \leq 2.3$
- **Medium Exam Target:** $2.3 < \mu_S \leq 3.8$
- **Hard Exam Target:** $3.8 < \mu_S \leq 5.0$

The system evaluates each candidate set S against these criteria. If a sample does not meet the target within 100 iterations, the set with the minimum distance to the target center is selected.

3.3 Component Level Implementation Details

To provide a comprehensive understanding of the system's backend, this section details the key functions and their operational logic within the Python modules. The system is composed of four primary modules, each handling a specific aspect of the exam generation lifecycle.

3.3.1 Data Persistence Layer (database.py)

The database module acts as the interface between the Streamlit frontend and the SQLite backend. It abstracts raw SQL queries into Python functions to prevent SQL injection and ensure data consistency.

- **Function:** `get_questions_by_criteria(subject, difficulty)`
 - **Input:** A subject string (e.g., "Mathematics") and a difficulty level ("Easy").
 - **Output:** A list of `Question` objects.
 - **Logic:** This function constructs a dynamic SQL query. It uses parameterized queries (?) to safely filter records. If a parameter is `None`, it is ignored, allowing for broad searches.
- **Function:** `add_question(question_obj)`
 - **Input:** A `Question` data class instance.
 - **Logic:** Performs an `INSERT` operation. It handles the serialization of the `options` list into a JSON string before storing it in the `TEXT` column of the database.

3.3.2 Data Ingestion Module (importer.py)

This module is responsible for parsing external data sources. It utilizes the `pandas` library to handle Excel files, which is robust against formatting errors.

- **Function:** `import_questions_from_excel(file)`
 - **Input:** A binary file stream (uploaded .xlsx file).
 - **Output:** A dictionary containing success counts and error logs.

– **Process:**

1. Reads the file into a DataFrame.
2. Normalizes column names (stripping whitespace).
3. Validates that required columns (Subject, Difficulty, Text) exist.
4. Iterates through rows, converts data types, and calls `add_question` for valid entries.
5. Catches exceptions (e.g., corrupt cells) and logs the specific row number for the user.

3.3.3 Document Construction Module (exporter.py)

The exportation logic is decoupled from the UI to support multiple formats. This module uses creating patterns to build documents in memory buffers.

- **Function:** `export_to_pdf(exam, filename)`
 - **Library:** `FPDF`
 - **Challenges Solved:** The standard `FPDF` library does not natively support UTF-8 characters (like Turkish 'ğ', 'ş'). The implementation includes a font fallback mechanism, attempting to load 'Arial.ttf' from the Windows font directory to ensure character support.
 - **Layout Logic:** It iterates through questions, using `multi_cell` for wrapping long question texts and identifying indentation for options.
- **Function:** `export_to_docx(exam)`
 - **Library:** `python-docx`
 - **Structure:**
 1. Adds the Exam Title as a Header 0.
 2. Loops through questions, adding them as paragraphs.
 3. If the question is "Multiple Choice", it parses the JSON options and adds them with the `List Bullet` style.
 4. Inserts a `Page Break`.
 5. Generates a synchronized "Answer Key" on the final page using the same index numbers.

3.3.4 Object Models (models.py)

To maintain type safety and code clarity, the system uses Python Data Classes ('@data-class') instead of raw dictionaries.

```
1 @dataclass
2 class Question:
3     id: Optional[int]
4     text: str
5     subject: str
6     difficulty: str # 'Easy', 'Medium', 'Hard'
7     q_type: str # 'Multiple Choice', 'Classic'
8     options: Optional[str] = None
9     answer: Optional[str] = None
```

Listing 1: Data Class Definitions

Using this structure ensures that all modules (Database, UI, Generator) speak a common language, reducing runtime 'KeyError' exceptions significantly.

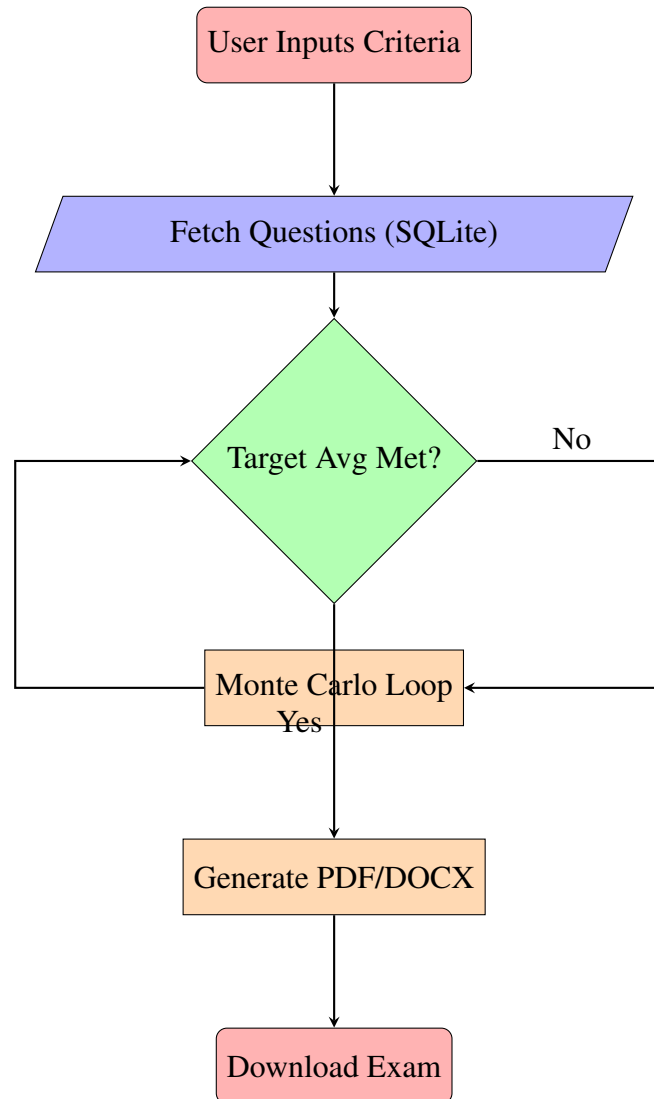


Figure 3.1: System Logic Flowchart: Visual representation of the question selection and export process.

4 APPLICATION AND SYSTEM DESIGN

This section describes the technical architecture, database schema, and the implementation of the exam generation workflow using Python and Streamlit [3].

4.1 Database Design and Schema

The system relies on a relational SQLite database to store question banks and exam logs. The choice of SQLite ensures portability, allowing the entire application to run locally without a dedicated database server.

The core data structure is defined in the `questions` table, which stores metadata such as difficulty level, question type, and options for multiple-choice questions. The SQL definition used in the `database.py` module is shown in Listing 1.

```
1 CREATE TABLE IF NOT EXISTS questions (  
2     id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     text TEXT NOT NULL,  
4     subject TEXT NOT NULL,  
5     difficulty TEXT NOT NULL,      -- 'Easy', 'Medium', 'Hard'  
6     q_type TEXT NOT NULL,         -- 'Multiple Choice', 'Classic'  
7     options TEXT,                 -- JSON array for choices  
8     answer TEXT  
9 );
```

Listing 2: SQL Schema definition for the Question Bank.

This schema supports flexibility. The `options` column utilizes a JSON string format, allowing the system to store a variable number of choices for test questions while keeping the field null for classic open-ended questions.

4.2 User Interface Implementation

The user interface is divided into two main modules: Data Entry and Exam Generation. To maximize usability, the design prioritizes a clean and responsive layout using Streamlit widgets.

The screenshot shows a web form titled "Elle Yeni Soru Ekle". It contains several input fields: "Ders (Örn: Matematik)" for subject, "Soru Tipi" (Question Type) with a dropdown menu currently showing "Multiple Choice", "Zorluk" (Difficulty) with a dropdown menu showing "Easy", and "Doğru Cevap" (Correct Answer). Below these is a large text area for "Soru Metni" (Question Text). A blue instruction box states: "Seçenekleri birbirinden ayırmak için her seçeneği yeni bir satıra yazın (Enter tuşu ile)." (To separate the options from each other, write each option on a new line (using the Enter key)). Below this is another text area for "Seçenekler" (Options). At the bottom left is a "Soru Ekle" (Add Question) button.

Figure 4.1: The Manual Data Entry Form. This interface allows instructors to categorize questions by subject and difficulty dynamically.

For bulk operations, an Excel-based import module is provided. This module parses ‘.xlsx’ files using the `pandas` library and maps columns to the database schema automatically, handling errors and missing fields gracefully.

Soru Yönetimi

The screenshot shows a web form titled "Excel'den Toplu Yükle". It contains a blue instruction box: "Soruları Excel dosyasından topluca yükleyebilirsiniz. Önce şablonu indirip doldurun." (You can bulk upload questions from an Excel file. First, download the template and fill it out.). Below this is a button "Örnek Şablonu İndir" (Download Example Template). Below that is a section titled "Excel Dosyanızı Buraya Sürükleyin" (Drag and drop your Excel file here) with a cloud icon and the text "Drag and drop file here" and "Limit 200MB per file • XLSX". To the right of this section is a "Browse files" button.

Figure 4.2: The Bulk Upload Interface. Users can drag and drop Excel files to populate the database instantly.

4.3 Algorithmic Implementation

The core logic of the system resides in the `generator.py` module. The system defines numerical weights for difficulty levels (*Easy* = 1, *Medium* = 3, *Hard* = 5) and attempts to find a subset of questions that matches a target average.

The code block below demonstrates the Monte Carlo selection logic implemented in the project. It iteratively samples questions to find a set that satisfies the difficulty criteria.

```
1 def generate_exam(title, criteria_list):
2     selected_questions = []
3
4     for criteria in criteria_list:
5         subject = criteria.get('subject')
6         # ... (Retrieving criteria)
7
8         # 1. Fetch ALL questions for this subject
9         all_questions = get_questions_by_criteria(subject, None)
10
11        # 2. Weighted Selection Algorithm
12        target_min, target_max = TARGET_RANGES.get(target_diff, (1.0,
13        5.0))
14
15        # Monte Carlo Simulation Loop
16        for _ in range(100):
17            sample = random.sample(all_questions, count)
18            avg = calculate_average_difficulty(sample)
19
20            if target_min <= avg <= target_max:
21                selected_questions.extend(sample)
22                found_match = True
23                break
24
25            # Keep track of the "closest" sample
26            dist = abs(avg - (target_min + target_max)/2)
27            if dist < best_avg_dist:
28                best_avg_dist = dist
29                best_sample = sample
30
31        return Exam(..., questions=selected_questions)
```

Listing 3: Monte Carlo Selection Logic from `generator.py`

This approach ensures that if a perfect mathematical match is not found within 100 iterations, the system gracefully degrades to the "closest possible" difficulty configuration, preventing runtime errors.

4.4 Document Generation (Export Module)

One of the key requirements was the ability to export exams to editable formats. The `exporter.py` module utilizes the `python-docx` library to construct a formatted Word document.

```
1 def export_to_docx(exam: Exam, filename: str = None):
2     doc = Document()
3     doc.add_heading(exam.title, 0)
4
5     for i, q in enumerate(exam.questions, 1):
6         doc.add_paragraph(f"{i}. {q.text} ({q.difficulty})")
7
8         if q.q_type == 'Multiple Choice' and q.options:
9             options = json.loads(q.options)
10            for opt in options:
11                doc.add_paragraph(f"- {opt}", style='List Bullet')
12
13    # --- Generate Answer Key on New Page ---
14    doc.add_page_break()
15    doc.add_heading("CEVAP ANAHTARI", level=1)
16
17    for i, q in enumerate(exam.questions, 1):
18        doc.add_paragraph(f"{i}. {q.answer}")
19
20    return buffer
```

Listing 4: Word Document Generation Function

The output of this function is a professional-grade document that includes both the question sheet and a separate answer key page.

Final Sınavı

1. Osmanlı'nın kurucusu kimdir? (Easy)

- Osman Bey
- Orhan Bey
- Fatih

2. Türkiye Cumhuriyeti ne zaman kuruldu? (Easy)

- 1923
- 1881
- 1453
- 1071
- 1213

3. Kurtuluş Savaşı hangi yıl başladı? (Medium)

- 1919
- 1920
- 1921

Figure 4.3: The final output generated by the system, showing the question structure and format.

5 CONCLUSION

This project successfully demonstrates the development of an automated exam generation system that bridges the gap between static question banks and pedagogical requirements. By implementing a Monte Carlo-based selection algorithm, the system ensures that generated exams are not only random but also statistically aligned with target difficulty levels.

The integration of Streamlit provided a highly accessible interface, while the SQLite back-end ensured data integrity and portability. The dual-export functionality (PDF and DOCX) addresses the practical needs of academic environments by providing both finalized and editable outputs.

5.1 Future Work

Future iterations of this project could incorporate several advanced features:

- **AI-Powered Question Generation:** Integrating Large Language Models (LLMs) to automatically generate new questions based on provided lecture notes.
- **Analytics Dashboard:** A module to track student performance and adjust question difficulty weights automatically based on historical success rates.
- **Cloud Integration:** Migration to a cloud-based database (e.g., PostgreSQL) to support multi-user environments and larger question repositories.

References

- [1] N. Metropolis and S. Ulam. The Monte Carlo method. *Journal of the American Statistical Association*, 44(247):335–341, 1949.
- [2] G. Van Rossum and F. L. Drake. *Python 3.12 and SQLite3 Database Integration*. Python Software Foundation, 2024.
- [3] Streamlit Team. Streamlit documentation: Building web applications with Python, 2025. Accessed: 2026-01-06.

RESUME

Name Surname: Demir KİRACIOĞLU

Place of Birth: Konak

Date of Birth: 12.12.2003

High School: Buca İnci-Özer Tırnaklı Fen Lisesi

Education: Yıldız Technical University, Mathematical Engineering